



به نام خدا



پروژه پنجم آزمایشگاه سیستم عامل

(مدیریت حافظه در xv6)

طراحان: هاتف رضایی، آرین فیروزی، علی عشقی موحد

در این پروژه شیوه مدیریت حافظه در سیستم عامل xv6 بررسی شده و قابلیت‌هایی به آن افزوده خواهد شد. در ادامه ابتدا مدیریت حافظه به طور کلی در xv6 معرفی شده و در نهایت صورت آزمایش شرح داده خواهد شد.

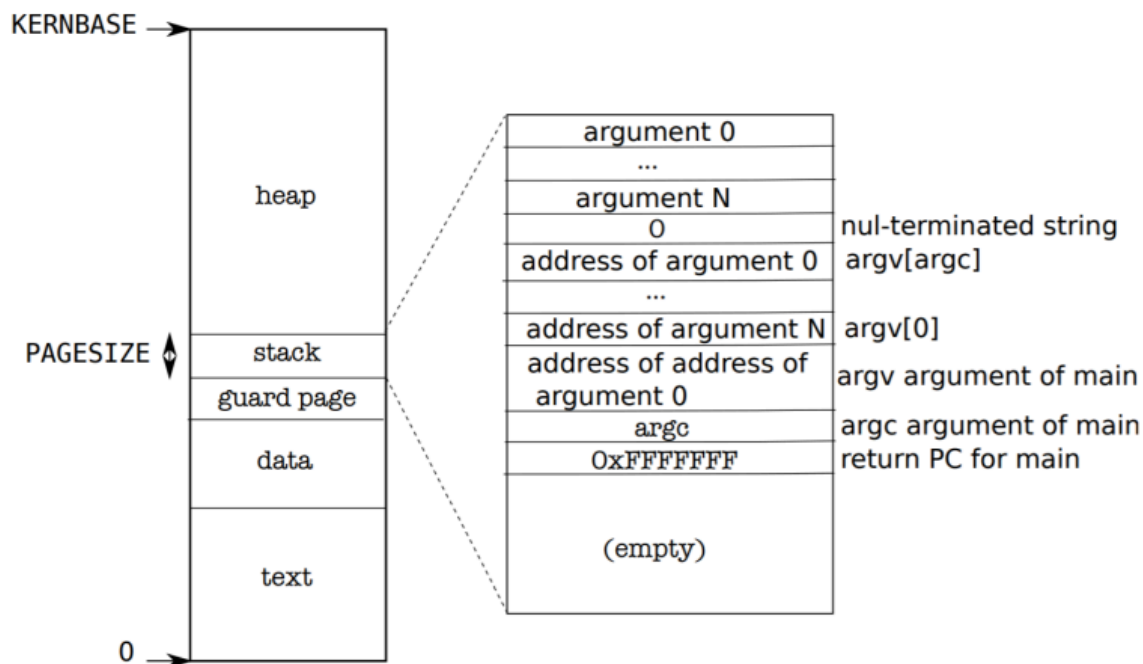
مقدمه

یک برنامه، حین اجرا تعامل‌های متعددی با حافظه دارد. دسترسی به متغیرهای ذخیره شده و فراخوانی توابع موجود در نقاط مختلف حافظه مواردی از این ارتباط‌ها می‌باشد. معمولاً کد منبع دارای آدرس نبوده و از نمادها برای ارجاع به متغیرها و توابع استفاده می‌شود. این نمادها توسط کامپایلر و پیونددهنده¹ به آدرس تبدیل خواهد شد. حافظه یک برنامه سطح کاربر شامل بخش‌های مختلفی مانند کد، پشته² و هیپ³ است. این ساختار برای یک برنامه در xv6 در شکل زیر نشان داده شده است:

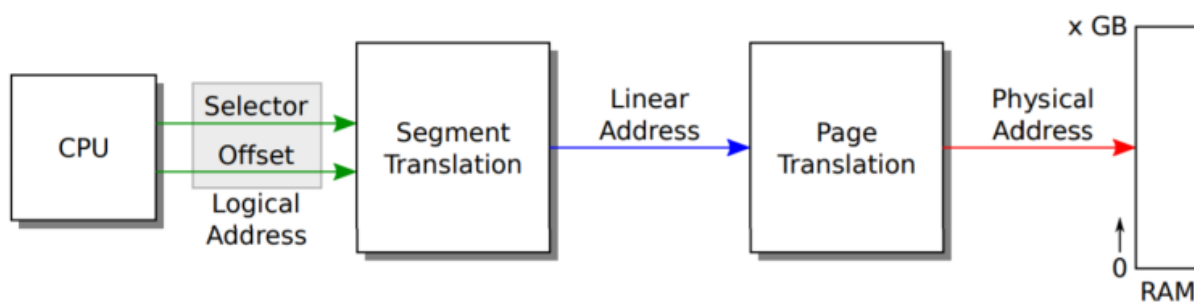
¹ Linker

² Stack

³ Heap



(۱) راجع به مفهوم ناحیه مجازی^۴ در لینوکس به طور مختصر توضیح داده و آن را با xv6 مقایسه کنید. صفحه های محافظ^۵ به چه منظور استفاده میشوند؟ همان طور که در آزمایش یک ذکر شد، در مد محافظت شده^۶ در معماری x86 هیچ کدی (اعم از کد هسته یا کد برنامه سطح کاربر) دسترسی مستقیم به حافظه فیزیکی^۷ نداشته و تمامی آدرس های برنامه از خطی^۸ به مجازی^۹ و سپس به فیزیکی تبدیل می شوند. این نگاشت در شکل زیر نشان داده شده است:



^۴ Virtual Memory Area

^۵ Guard Page

^۶ Protected Mode

^۷ Physical Memory

^۸ Linear

^۹ Virtual

به همین منظور، هر برنامه یک جدول اختصاصی موسوم به جدول صفحه¹⁰ داشته که در حین فرآیند تعویض متن¹¹ بارگذاری شده و تمامی دسترسی‌های حافظه (اعم از دسترسی به هسته یا سطح کاربر) توسط آن برنامه توسط این جدول مدیریت می‌شود.

به علیت عدم استفاده صریح از قطعه‌بندی در بسیاری از سیستم‌عامل‌های مبتنی بر این معماری، می‌توان فرض کرد برنامه‌ها از صفحه‌بندی¹² و لذا آدرس مجازی استفاده می‌کنند. علت استفاده از این روش مدیریت حافظه در درس تشریح شده است. به طور مختصر می‌توان سه علت عمده را برشمرد:

(۱) **ایزوله‌سازی پردازنده‌ها از یکدیگر و هسته از پردازنده‌ها:** با اجرای پردازنده‌ها در فضاهای آدرس¹³ مجزا، امکان دسترسی یک برنامه مخرب به حافظه برنامه‌های دیگر وجود ندارد. ضمن این که با اختصاص بخش مجزا و ممتازی از هر فضای آدرس به هسته امکان دسترسی محافظت‌نشده پردازنده‌ها به هسته سلب می‌گردد.

(۲) **ساده‌سازی ABI سیستم‌عامل:** هر پردازنده می‌تواند از یک فضای آدرس پیوسته (از آدرس مجازی صفر تا چهار گیگابایت در معماری x86) به طور اختصاصی استفاده نماید. به عنوان مثال کد یک برنامه در سیستم‌عامل

لینوکس در معماری x86 همواره (در صورت عدم استفاده از تصادفی‌سازی چینش فضای آدرس¹⁴) از آدرس 0x08048000 آغاز شده و نیاز به تغییر در آدرس‌های برنامه‌ها متناسب با وضعیت جاری تخصیص حافظه فیزیکی نمی‌باشد.

(۳) **استفاده از جابه‌جایی حافظه:** با علامت‌گذاری برخی از صفحه‌ها کم استفاده (در جدول صفحه) و انتقال آن‌ها به دیسک، حافظه بیشتری در دسترس خواهد بود. به این عمل جابه‌جایی حافظه¹⁵ اطلاق می‌شود.

¹⁰ Page Table

¹¹ Context Switch

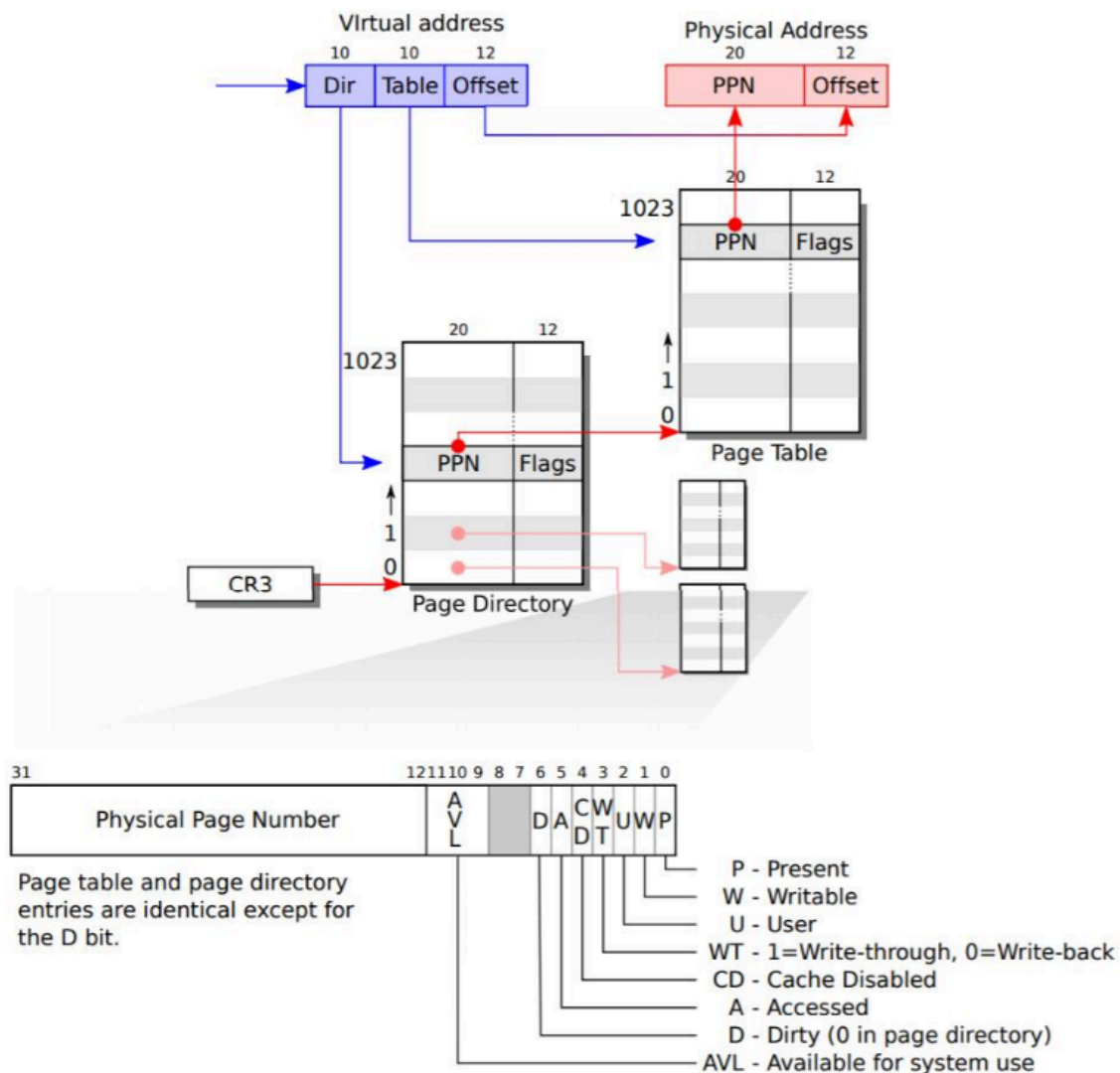
¹² Paging

¹³ Address Spaces

¹⁴ Address Space Layout Randomization

¹⁵ Memory Swapping

ساختار جدول صفحه در معماری x86 (در حالت بدون گسترش آدرس فیزیکی¹⁶ (PAE) و گسترش اندازه صفحه¹⁷ (PSE)) در شکل زیر نشان داده شده است.



هر آدرس مجازی توسط اطلاعات این جدول به آدرس فیزیکی تبدیل می‌شود. این فرآیند، سخت‌افزاری بوده و سیستم‌عامل به طور غیرمستقیم با پر کردن جدول، نگاشت را صورت می‌دهد. جدول صفحه دارای سلسله مراتب دو سطحی بوده که به ترتیب Page Directory و Page Table نام دارند. هدف از ساختار سلسله‌مراتبی کاهش مصرف حافظه است.

(۲) چرا ساختار سلسله‌مراتبی منجر به کاهش مصرف حافظه می‌گردد؟

(۳) محتوای هر بیت یک مدخل (۳۲ بیتی) در هر سطح چیست؟ چه تفاوتی میان آن‌ها وجود دارد؟

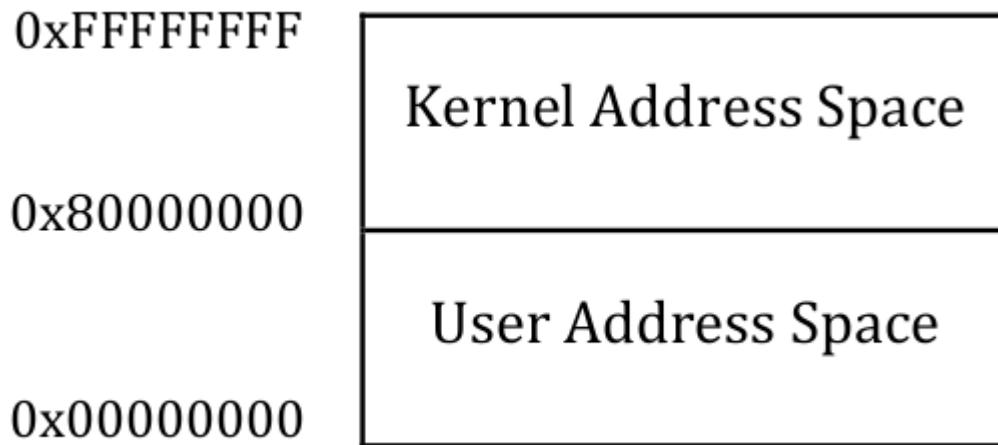
¹⁶ Physical Address Extension

¹⁷ Page Size Extension

مدیریت حافظه در xv6

ساختار فضای آدرس در xv6

در xv6 نیز مد اصلی اجرای پردازنده، مد محافظت‌شده و ساز و کار اصلی مدیریت حافظه صفحه‌بندی است. به این ترتیب نیاز خواهد بود که پیش از اجرای هر کد، جدول صفحه آن در دسترس پردازنده قرار گیرد. کدهای اجرایی در xv6 شامل کد پردازنده‌ها (کد سطح کاربر) و ریسسه هسته متناظر با آن‌ها و کدی است که در آزمایش یک، کد مدیریت‌کننده نام‌گذاری شد.¹⁸ آدرس‌های کد پردازنده‌ها و ریسسه هسته آن‌ها توسط جدول صفحه‌ای که‌هاشاره‌گر به ابتدای Page Directory آن در فیلد pgdir



از ساختار (proc خط ۲۳۳۹) قرار دارد، نگاشت داده می‌شود. نمای کلی ساختار حافظه مجازی متناظر با جدول این دسته در شکل زیر نشان داده شده است:

دو گیگابایت پایین جدول صفحه مربوط به اجزای مختلف حافظه سطح کاربر پردازنده است. دو گیگابایت بالای جدول صفحه مربوط به اجزای ریسسه هسته پردازنده بوده و در تمامی پردازنده‌ها یکسان است. آدرس تمامی متغیرهایی که در هسته تخصیص داده می‌شوند در این بازه قرار می‌گیرد. جدول صفحه کد مدیریت‌کننده هسته، دو گیگابایت پایینی را نداشته (نگاشتی در این بازه ندارد) و دو گیگابایت بالای آن دقیقاً شبیه به پردازنده‌ها خواهد بود. زیرا این کد، همواره در هسته اجرا شده و پس از بوت غالباً، در اوقات بیکاری سیستم اجرا می‌شود.

¹⁸ بحث مربوط به پس از اتمام فرآیند بوت است. به عنوان مثال، در بخشی از بوت، از صفحات چهار مگابایتی استفاده شد که از آن صرف نظر شده است.

کد مربوط به ایجاد فضاهای آدرس در xv6

فضای آدرس کد مدیریت‌کننده هسته در حین بوت، در تابع `main` ایجاد می‌شود. به این ترتیب که تابع `kvmalloc` فراخوانی شده (خط ۱۲۲۰) و به دنبال آن تابع `setupkvm` متغیر `kpgdir` را مقداردهی می‌نماید (خط ۱۸۴۲). به طور کلی هر زمان نیاز به مقداردهی ساختار فضای آدرس هسته باشد، از `setupkvm` استفاده خواهد شد. با بررسی تابع `setupkvm` (خط ۱۸۱۸) می‌توان دریافت که در این تابع، ساختار فضای آدرس هسته بر اساس محتوای آرایه `kmap` (خط ۱۸۰۹) دیده می‌شود.

۴) تابع `kalloc` چه نوع حافظه‌ای تخصیص می‌دهد؟ (فیزیکی یا مجازی)

۵) تابع `mappages` چه کاربردی دارد؟

۶) آیا تمام قسمت‌های حافظه مجازی از آدرس صفر تا `proc->sz` از طریق کاربر قابل استفاده اند؟

فضای آدرس مجازی نخستین برنامه سطح کاربر (`initcode`) نیز در تابع `main` ایجاد می‌گردد. به طور دقیق‌تر تابع `userinit` (خط ۱۲۳۵) فراخوانی شده و توسط آن ابتدا نیمه هسته فضای آدرس با اجرای `setupkvm` (خط ۲۵۲۸) مقداردهی خواهد شد. نیمه سطح کاربر نیز توسط تابع `inituvm` ایجاد شده تا کد برنامه نگاشت داده شود. فضای آدرس باقی‌پرده‌ها در ادامه اجرای سیستم توسط توابع `fork` یا `exec` مقداردهی می‌شوند. به این ترتیب که هنگام ایجاد پرده فرزند توسط `fork` با فراخوانی تابع `copyuvm` (خط ۲۵۹۲) فضای آدرس نیمه هسته ایجاد شده (خط ۲۰۴۲) و سپس فضای آدرس نیمه کاربر از والد کپی می‌شود. این کپی با کمک تابع `walkpgdir` (خط ۲۰۴۵) صورت می‌پذیرد.

۷) راجع به تابع `walkpgdir` توضیح دهید. این تابع چه عمل سخت‌افزاری را شبیه‌سازی می‌کند؟

وظیفه تابع `exec` اجرای یک برنامه جدید در ساختار بلوک کنترلی پرده^{۱۹} یک پرده موجود است. معمولاً پس از ایجاد فرزند توسط `fork` فراخوانده شده و کد، داده‌های ایستا، پشته و هیپ برنامه جدید را در فضای آدرس فرزند ایجاد می‌نماید. بدین ترتیب با اعمال تغییراتی در فضای آدرس موجود، امکان اجرای یک برنامه جدید فراهم می‌شود. روش متداول `Shell` در سیستم‌های مبتنی بر یونیکس از جمله `xv6` برای اجرای برنامه‌های جدید مبتنی بر `exec` است. `Shell` پس از دریافت ورودی و فراخوانی `fork1` تابع `runcmd` را برای اجرای دستور ورودی، فراخوانی می‌کند (خط ۸۷۲۴). این تابع نیز در نهایت تابع `exec` را فراخوانی می‌کند (خط ۸۶۲۶). چنانچه در آزمایش یک مشاهده شد، خود `Shell` نیز در حین بوت با فراخوانی سیستمی `sys_exec` (خط ۸۴۱۴) و به دنبال آن `exec` ایجاد شده و فضای آدرسش به جای فضای آدرس نخستین پرده `initcode` چیده می‌شود. در پیاده‌سازی `exec` مشابه قبل `setupkvm` فراخوانی شده (خط ۶۶۳۷) تا فضای آدرس هسته تعیین گردد. سپس با فراخوانی `allocuvm` فضای مورد نیاز برای کد و داده‌های برنامه جدید (خط ۶۶۵۱) و صفحه محافظ و پشته (خط ۶۶۶۵) تخصیص داده می‌شود. دقت شود تا این مرحله تنها تخصیص صفحه صورت گرفته و باید این فضاها در ادامه توسط توابع مناسب با داده‌های مورد نظر پر شود (به ترتیب خطوط ۶۶۵۵ و ۶۶۸۶).

^{۱۹} Process Control Block

۸ (توابع allocvm و mappages که در ارتباط با حافظه‌ی مجازی هستند را توضیح دهید.

۹ (شیوه‌ی بارگذاری²⁰ برنامه در حافظه توسط فراخوانی سیستمی exec را شرح دهید.

شرح پروژه

هدف پروژه:

شما در درس با ساختار مانیتور در مدیریت حافظه اشتراکی و همزمانی داده‌ای آشنا شدید. در این پروژه، هدف پیاده‌سازی قابلیت حافظه‌ی اشتراکی با استفاده از مانیتور و جدول در هسته‌ی سیستم‌عامل xv6 است تا کاربر بتواند آسان‌تر به حافظه اشتراکی دسترسی خواندن و نوشتن پیدا کند. این قابلیت امکان به‌اشتراک‌گذاری یک فضای حافظه بین چند پردازش را فراهم می‌کند، به‌گونه‌ای که پردازش‌ها بتوانند به‌صورت هم‌راند از فضای اشتراکی استفاده کنند. همچنین از حافظه اشتراکی برای IPC نیز استفاده می‌شود.

الزامات پیاده‌سازی:

1. ساختار داده مدیریت حافظه‌ی اشتراکی

یک جدول مرکزی برای نگهداری اطلاعات مربوط به بخش‌های حافظه‌ی اشتراکی تعریف و پیاده‌سازی کنید. هر ورودی این جدول شامل اطلاعات زیر است:

- شماره یکتای حافظه‌ی اشتراکی
- اشاره‌گر به آدرس فیزیکی شروع حافظه
- شمارنده‌ی تعداد پردازش‌های متصل به این حافظه

توجه داشته باشید که این جدول در واقعیت باید با قفل پیاده شود، اما در این تمرین برای انجام عملیات بدون قفل استفاده خواهد شد بنابراین می‌توانید آن را به صورت یک جدول با تنها یک صفحه‌ی حافظه‌ی مشترک پیاده سازی کنید.

2. افزودن فراخوانی‌های سیستمی جدید

برای ایجاد و از بین بردن صفحه‌های حافظه دو فراخوانی سیستمی جدید به سیستم‌عامل اضافه شود:

- `open_shared_mem(int id):`

²⁰ Load

این تابع وظیفه دارد حافظه‌ای با آیدی مشخص را برای پردازش فراهم کند. در صورتی که حافظه‌ی مورد نظر از قبل اختصاص داده نشده باشد، باید فرآیند ایجاد و ثبت آن در جدول حافظه‌ی اشتراکی انجام شود. همچنین باید اشاره‌گر به ابتدای حافظه‌ی اشتراکی به فضای کاربر بازگردانده شود.

- `close_shared_mem(int id):`

این فراخوانی مسئول آزادسازی حافظه‌ی اشتراکی از دید پردازشی فراخواننده است. در صورتی که تعداد مراجع به حافظه به صفر برسد، باید ورودی مربوطه از جدول حذف شود.

3. پیاده سازی monitor

در این بخش به فراخوانی‌های سیستمی زیر باید ایجاد شوند:

- `monitor_init( int shared_mem_id, int* initial_value, int size_value)`
 - این فراخوانی سیستمی یک shared memory با آیدی داده شده را ایجاد و مقدار اولیه را در آن می‌ریزد
- `monitor_increase_all_elems(int shared_mem_id)`
 - این فراخوانی سیستمی مقدار همه ی المان های shared memory را یکی زیاد می کند
- `monitor_close_shared_mem(int shared_mem_id)`
 - این فراخوانی سیستمی shared memory را می بندد و حافظه مربوط به آن را آزاد میکند
- `monitor_read_shared_mem(int shared_memory_id, int* data)`
 - این فراخوانی سیستمی اشاره گری به یک کپی از حافظه ی مشترک شده را بر روی آرایه ای که از کاربر گرفته مینویسد تا کاربر بتواند بدون تغییر حافظه اصلی آن را بخواند

در این مانیتور، مقادیر آرایه در یک صفحه حافظه ذخیره خواهند شد. لازم است قبل فراخوانی تابعی که در قسمت 2 پیاده کردید، این نکته را در نظر داشته باشید که اندازه آرایه از حداکثر طول صفحه (که در xv6 ذکر شده است) بیشتر نباشد. در غیر این صورت یک مقدار برگردانید که مشخص شود عملیات ناموفق بوده. همچنین داخل مانیتور باید عملیات با استفاده از یک قفل mutex انجام شوند تا همزمانی دو عملیات یکپارچگی داده ای را خراب نکند.

10) پیاده سازی مانیتور ها معمولا در برگیرنده ی مفهومی به اسم متغییر شرطی²¹ برای signaling است، اما در این پروژه از شما خواسته نشده چنین ساز و کاری پیاده کنید. این مکانیزم را توضیح دهید و شرح دهید پیاده سازی آن در معماری xv6 چه مشکلاتی ممکن است به همراه داشته باشد؟

4. برنامه ی سطح کاربر

برای این بخش برنامه سطح کاربری بنویسید که دو پردازنده مقدار یک حافظه مشترک در بر دارنده یک مقدار صحیح است را 1000 بار زیاد می کنند. سپس این برنامه یک بار با استفاده از مانیتور و یک بار بدون استفاده از هیچ مکانیزم کنترل همزمانی پیاده کنید. نتیجه خروجی دو برنامه را مقایسه و علت تفاوت آنها را در گزارش بیان کنید.

²¹ condition variable

نکات پیاده سازی

- پروژه خود را در Github یا Gitlab پیش برده و در نهایت یک نفر از اعضای گروه کدها را به همراه پوشه git و گزارش زیپ کرده و در سامانه با فرمت OS-Lab4-<SID1>-<SID2>-<SID3>.zip آپلود نمایید.
- رعایت نکردن مورد فوق کسر نمره را به همراه خواهد داشت.
- بخش خوبی از نمره شما را پاسخ دهی به سوالات مطرح شده، تشکیل میدهد که به شما در درک نحوه کارکرد xv6 و پیاده سازی قسمت سوالات کمک میکند.
- پاسخ سوالها را تا حد ممکن کوتاه بنویسید.
- همه افراد می بایست به پروژه مسلط باشند و نمره تمامی اعضای گروه لزوما یکسان نیست.
- تمامی مواردی که در جلسه توجیهی گروه اسکایپ و فروم درس مطرح می شوند، جزئی از پروژه خواهند بود در صورت وجود هرگونه سوال یا ابهام میتوانید با ایمیل دستیاران مربوطه یا گروه اسکایپی درس در ارتباط باشید.
- این تمرین صرفا برای یادگیری شما طرح شده است.
- در صورت محرز شدن تقلب در تمرین مطابق با قوانین درس برخورد خواهد شد.

موفق باشید